UNIVERSITÀ DEGLI STUDI DI PISA

Decentralised Lending Service

PEER TO PEER SYSTEMS AND BLOCKCHAINS

Project

Student
Simone Infantino
123

Student
Antonio Tamborrino
456

Academic Year
2025/2026

Contents

Chapter 1

Intro and Implementation Choices

1.1 Smart Contract Implementation

The decentralized lending platform is implemented in Solidity and developed using the Hardhat framework with the Viem interaction layer. Its architecture consists of three smart contracts: `LendingService`, which manages the protocol; `Loan`, which represents individual loans; and `BitcoinOracle`, which provides Bitcoin collateral information. This separation improves modularity and simplifies maintenance.

1.2 Lending Service Architecture

The `LendingService` contract manages liquidity, loan proposals, voting, contributor balances, and oracle interaction. Rather than storing all loans internally, each approved proposal deploys an independent `Loan` contract containing its own state. Contributor funds are divided into available deposits and locked liquidity, ensuring that only uncommitted funds can be withdrawn. A minimum deposit is required to participate, and inactive contributors are removed from the registry to reduce storage costs.

1.3 Loan Proposal and Voting Mechanism

Applicants submit loan proposals specifying the requested amount, interest rate, duration, and Bitcoin collateral address. Proposals remain open for a fixed voting period during which contributors vote with a weight proportional to their available liquidity.
A proposal is approved only if sufficient liquidity is available, the Bitcoin balance provided by the oracle satisfies the collateral requirement, and approval votes exceed rejection votes. Successful proposals generate a new `Loan` contract and lock contributor funds proportionally.

1.4 Loan Contract Design

Each `Loan` contract represents a single lending agreement and transfers the requested funds to the borrower immediately after deployment.
The contract stores immutable loan parameters, including the borrower, principal, interest rate, duration, collateral ratio, and Bitcoin address. Repayments can be partial or complete and are distributed proportionally between principal and interest, with contributor rewards and compensation handled automatically.
The contract also stores the contributors participating in the loan together with their locked amounts, allowing repayments to be distributed proportionally.

1.5 Bitcoin Oracle Integration

The `BitcoinOracle` contract supplies Bitcoin balance information required to validate collateral. Applicants may request an update by paying a minimum fee, while an off-chain oracle service scans the Bitcoin blockchain and the oracle owner submits the updated balances on-chain.
This design separates on-chain logic from external data acquisition, with only the oracle owner authorized to update stored balances. The immutable deployment block is also stored to allow the off-chain service to resume synchronization from the correct point.

1.6 Security and Administrative Choices

The contracts adopt a two-step ownership transfer mechanism for both the lending service and the oracle to prevent accidental administrative changes. Both contracts also support controlled termination, while the lending service additionally enables migration to a successor contract once all active loans have been completed.
The lending service also supports controlled protocol migration by allowing termination only after all active loans are completed. During migration, the remaining Ether balance is transferred to a designated successor contract. Callback functions enable communication between `Loan` and `LendingService` while preserving modularity.
Finally, the project includes intentionally vulnerable contracts and a reentrancy attacker used to evaluate security measures and validate protection against reentrancy attacks.

Chapter 2

Reentrancy and Security Discussion

2.1 Gas Evaluation

Gas consumption was evaluated using the Hardhat testing environment. The analysis focused on the main protocol operations, including deposits, proposal approval, loan creation, repayments, and compensation claims.
To improve efficiency, the contracts minimize storage writes and use `immutable` variables for loan parameters that never change after deployment. The modular architecture, where each approved proposal deploys a dedicated `Loan` contract, increases deployment cost but simplifies the main contract and improves maintainability.
Some functions iterate over the contributor list to compute voting weights, available liquidity, and fund allocation, causing gas costs to grow with the number of contributors. This trade-off was considered acceptable for the project. Oracle integration is also gas-efficient because Bitcoin data are processed off-chain and only the required balances are stored on Ethereum.

2.2 Reentrancy Protection

The project includes both a secure implementation (`LendingService.sol`) and an intentionally vulnerable version (`LendingServiceVulnerable.sol`) used to demonstrate a reentrancy attack.
The vulnerability affects the `claim_compensation()` function, where the vulnerable contract transfers Ether before updating its internal state. A malicious contract can exploit this behavior by re-entering the function from its `receive()` callback while the previous execution is still in progress, allowing multiple compensation claims before the contributor's accounting information is updated.
The secure implementation follows the Checks–Effects–Interactions pattern by updating the contract state before performing external transfers, preventing recursive calls from reusing outdated accounting information.
The attack is implemented in `ReentrancyAttacker.sol` and validated through dedicated Hardhat tests, which confirm that the vulnerable contract can be exploited while the protected implementation resists the same attack.

2.3 Discussion of Malicious Strategies

Besides reentrancy, other potential attack scenarios were considered during the design of the protocol.
The voting mechanism is based on contributors' available liquidity, making voting influence proportional to the amount of funds supplied rather than assigning equal voting power to each address. This aligns voting power with the economic stake committed to the protocol.
Loan approval also depends on multiple independent conditions. In addition to a positive vote, sufficient liquidity must be available and the Bitcoin collateral value provided by the oracle must satisfy the required collateralization threshold. Combining these conditions prevents proposals from being approved solely through voting.
The protocol relies on a trusted oracle owner to submit correct Bitcoin balance updates. Since blockchain data are obtained off-chain, the oracle remains a trusted component whose correctness directly affects collateral evaluation.
Although these measures improve the robustness of the system, they do not eliminate every possible attack vector. As with most decentralized applications, the overall security also depends on the correct behaviour of external components and administrative accounts.

Chapter 3

User Manual

3.1 Requirements

The project was developed and tested using the following software versions:

- **Geth**: 1.13.15-stable-c5ba367e
- **Node.js**: v22
- **Hardhat**: v3.9.0
- **Solidity**: v0.8.28
- **Apache Maven**: v3.9.9

Before executing the project, the Bitcoin blockchain blocks required by the off-chain scanner must be manually copied into the `bt_chain_blocks` directory located in the project root.

3.2 Project Structure

The project is organized into independent modules that separate blockchain data, smart contracts, off-chain services, deployment scripts, and utilities.

- **bt_chain_blocks/**: local Bitcoin blockchain used by the off-chain scanner.
- **chain/**: local Ethereum configuration (genesis, accounts, and Geth data).
- **hardhat/**: Solidity smart contracts and deployment scripts.
- **offchain-scanner/**: Java application that extracts UTXO data from the Bitcoin blockchain.
- **python/**: Python scripts for deployment, oracle management, automated voting, and demonstration.
- **state/**: stores deployment state shared by the Python components.
- **tool.py**: orchestration script used to manage the entire system.
- **venv/**: Python virtual environment with project dependencies.

3.3 Orchestration Script

The `tool.py` script provides an interactive command line interface to automate the setup and execution of the system. The available operations are:

1. Run the Off-chain Scanner.
2. Create the Python virtual environment and install dependencies.
3. Initialize the local Ethereum blockchain.
4. Start the Geth node with mining and exposed HTTP JSON-RPC interface.
5. Create accounts and deploy contracts.
6. Start the Oracle Daemon.
7. Start the Auto Voter.
8. Run the lending demonstration.
9. Attach to the running Geth console.

The recommended execution order follows the list above. Long-running services (Geth, Oracle Daemon, and Auto Voter) must remain active while the system is running; therefore, each of them should be started in a separate terminal by executing `python3 tool.py` and selecting the corresponding option.